

Guía de arquitectura del sistema de facturación CFDI 4.0

Manual de estudio del sistema **SaaS de facturación electrónica CFDI 4.0** (México). Está escrito para que el desarrollador lo entienda, lo explique y lo **defienda**: cada decisión importante incluye el **porqué**, no solo el **qué**. Fuente única de las reglas: `ARQUITECTURA.md`. Si este documento y ese se contradicen, manda `ARQUITECTURA.md`.

Índice

1. Qué es el sistema
 2. El stack y los tres proyectos
 3. Cómo se comunican las partes
 4. El ciclo de una petición, de principio a fin
 5. Identidad y sesión (quién es quién)
 6. Autorización por permisos
 7. Cómo se guardan los datos (base de datos y aislamiento)
 8. Criptografía y secretos
 9. Cómo se monta el servicio (despliegue)
 10. Comandos útiles del servidor
 11. Recorridos de extremo a extremo
 12. Lo que quedó fuera del MVP
 13. Glosario
-

1. Qué es el sistema

Es un **SaaS** (software como servicio) de facturación electrónica para México. Emite comprobantes fiscales digitales por internet (**CFDI 4.0**, el esquema vigente del SAT).

Multiempresa: una **cuenta** contrata el servicio (por ejemplo “Refaccionaria García”) y dentro puede tener varias **empresas emisoras** (una llantera y una cementera del mismo dueño). Cada empresa aísla por completo sus clientes, productos, series de folios y comprobantes: lo que ve la llantera, no lo ve la cementera.

Quiénes lo usan: contadores y capturistas en jornadas largas, principalmente en escritorio, pero también en celular para consultar, reenviar y emitir una factura rápida.

Dos mitades funcionales:

- **Mitad A — plataforma:** cuentas, identidad (login, roles por permiso), catálogos del SAT, clientes, productos, series/folios, bolsa de timbres.
- **Mitad B — emisión:** motor de impuestos, generación de XML y PDF, timbrado contra el PAC, correo, listado de documentos, cancelación y complemento de pagos.

Fecha límite del MVP: **5 de diciembre de 2026.**

2. El stack y los tres proyectos

Capa	Tecnología
Interfaz	Blazor WebAssembly como PWA + MudBlazor
Servidor	ASP.NET Core Web API
Base de datos	SQL Server con EF Core 10
Lenguaje	.NET 10, C# 14
Logs	Serilog + Sentry
Impuestos / XML	Pantallas propias + esquemas XSLT del SAT
Timbrado	Integración con un PAC (proveedor autorizado) — hoy SW Sapien en sandbox

La solución se divide en **tres proyectos** (más uno de pruebas):

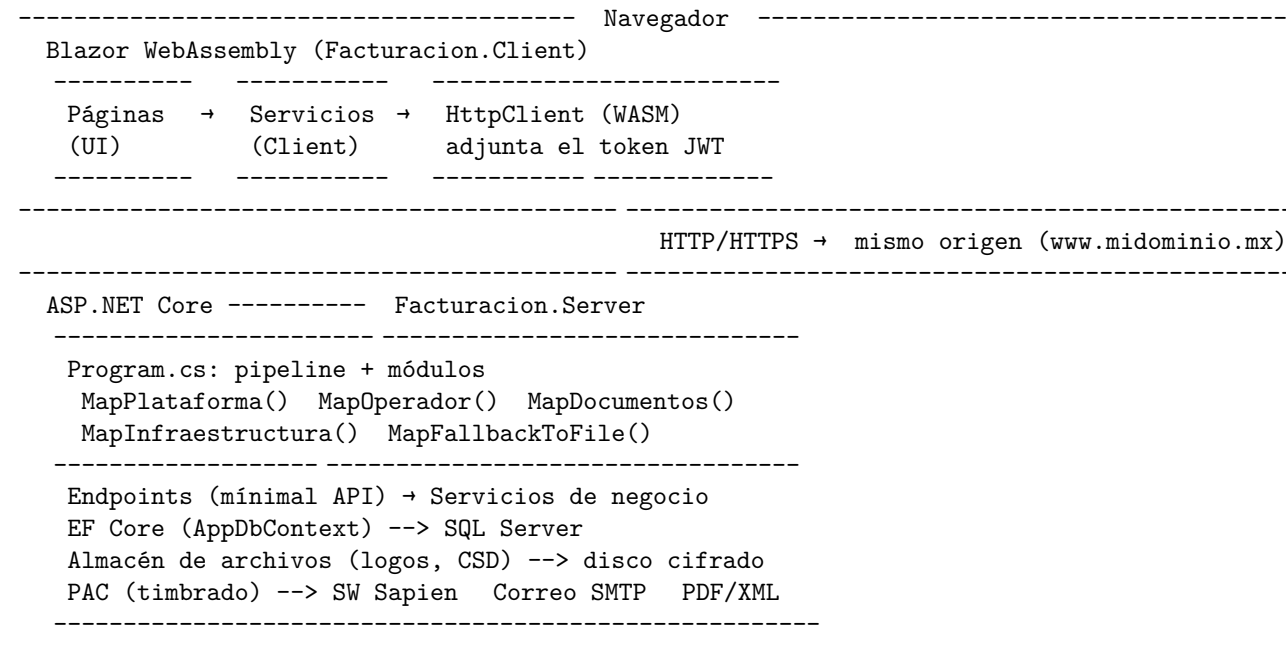
<code>Facturacion.Client</code>	Blazor WebAssembly (PWA)	→ corre en el NAVEGADOR
<code>Facturacion.Server</code>	ASP.NET Core Web API	→ corre en el SERVIDOR
<code>Facturacion.Shared</code>	biblioteca de clases	→ comparte DTOs y contratos
<code>tests/Facturacion.Pruebas</code>		→ pruebas obligatorias

Regla de oro del WebAssembly: el código del `Client` viaja al navegador y **cualquiera puede leerlo**. Por eso ninguna validación fiscal seria, ningún precio y ningún secreto viven en el cliente. Los precios los devuelve el servidor; el cliente solo manda el id de un paquete. La validación del cliente existe para dar retroalimentación rápida; la que cuenta es la del servidor y **se ejecuta siempre**.

La frontera congelada: `Facturacion.Shared/Contratos/` es la “interfaz” entre las dos mitades. Cambiar una firma ahí es un asunto formal: requiere acuerdo explícito y un commit dedicado. Es lo que permite que las dos mitades evolucionen sin romperse entre sí.

3. Cómo se comunican las partes

Es una sola aplicación web, no varias.



Puntos clave:

1. **Un solo origen.** El Server sirve la app de Blazor (`index.html`, el WASM y los estáticos) y además expone la API REST. Todo vive en el mismo dominio. Ventaja concreta: la cookie de refresh puede ser `SameSite=Strict` sin problemas de navegador.
2. **El cliente habla por HTTP con la API.** Cada página usa un servicio de `Client/Servicios/` que llama a los endpoints REST del Server (prefijo `/api/...`). Toda la “inteligencia” está en el servidor.
3. **Dos redes independientes en el cliente** (ver `Client/Program.cs`):
 - `CientesHttp.Api` — del **inquilino** (quien factura). Adjunta el JWT del inquilino.
 - `CientesHttp.ApiOperador` — del **operador** (el proveedor del SaaS). Adjunta el JWT del operador.
 - Están separadas a propósito: la credencial del proveedor **nunca** puede salir hacia un endpoint de inquilino y viceversa.
 - Además hay dos clientes “desnudos” (sin token) solo para el circuito de refresh.
4. **El Service Worker nunca cachea /api/.** La PWA cachea solo el

“caparazón” de la app. Si cacheara respuestas de la API, un catálogo del SAT viejo produciría comprobantes mal emitidos. Hay un endpoint `/api/version`: si la versión del servidor no coincide con la del cliente, la app muestra un aviso y fuerza recarga.

5. Toda ruta que no es de la API la atiende el Wasm (`MapFallbackToFile("index.html")`).

4. El ciclo de una petición, de principio a fin

Ejemplo: el capturista guarda un producto nuevo.

1. El capturista llena el formulario en una página Blazor.
2. La página llama a `ServicioDeProductos` (`Facturacion.Client`).
3. El `HttpClient` adjunta el JWT (Bearer) que está en memoria.
4. Va por HTTPS a `POST /api/productos`
5. El Server valida el JWT (firma, vigencia, audiencia) y la política de permiso (`configurar_empresa`).
6. Se arma el `AppDbContext` del request. Su "empresa actual" sale de los CLAIMS del token (`ContextoEmpresaHttp`) - nunca del cuerpo de la petición.
7. El servicio de negocio valida datos (clave SAT, RFC, etc.) y hace el `SaveChanges`.
8. El interceptor `SelladoDeEmpresaInterceptor` sella `EmpresaId` en el renglón nuevo.
9. SQL Server guarda; EF Core devuelve el resultado; el endpoint responde JSON.
10. La página actualiza la tabla y muestra el producto.

Estructura de carpetas del Server (hazla tuya para orientarte):

Data/	Entidades, configuraciones EF, <code>AppDbContext</code> , procedimientos asociados
Infra/	Correo, tenencia, idempotencia, bitácora, registro, almacén de archivos
Migrations/	Migraciones EF (generadas, no se editan a mano)
Modules/	
Plataforma/	Cuentas, auth, clientes, productos, series, timbres, catálogos SAT, tableros
Operador/	Panel del proveedor: operadores, paquetes, compras, membresías, cuentas
Documentos/	Emisión, impuestos, timbrado, PAC, salidas (XML/PDF), cancelación, pagos

5. Identidad y sesión (quién es quién)

El sistema tiene **dos identidades** que NO se mezclan:

	Inquilino (contrata y factura)	Operador (provee el SaaS)
Tabla	<code>AspNetUsers</code> (Identity)	<code>OperadoresPlataforma</code>
Claim en JWT	<code>emp, cta, usr, perm:*</code>	<code>opr, perm:*</code>
Audiencia JWT	<code>.../ normal</code>	<code>.../operador</code> (no intercambiables)

	Inquilino (contrata y factura)	Operador (provee el SaaS)
Cookie de refresh	refresh en <code>/api/auth</code>	refresh_op en <code>/api/operador/auth</code>
¿Se crea por pantalla?	Registro con verificación	Solo por consola (<code>--crear-operador</code>)

Reglas de sesión (no negociables):

- El access token JWT dura **15 minutos** y vive **solo en memoria** (nunca en `localStorage`).
- La sesión la mantiene una **cookie de refresh** `HttpOnly, Secure, SameSite=Strict`, que **se rota en cada uso**: el token usado se invalida y se emite uno nuevo. Si llega un token ya usado, se invalida **toda la familia** de sesiones — es la señal de que robaron la cookie.
- “Mantener sesión” = solo la duración de la cookie (12 h sin marcar, 30 días marcado). No se guarda nada más en el navegador.
- Al abrir la app hay un *gate*: intenta `POST /api/auth/refresh` con la cookie y no pinta nada hasta que resuelve (ni un parpadeo de “invitado”).
- Identity se usa **solo como almacén y hasher** — el JWT se emite con código propio.
- **La empresa activa es un claim del token**, y cambiar de empresa pide un token nuevo a `POST /api/auth/cambiar-empresa`. El cliente **jamás** manda un `empresaId` por ruta, query o cuerpo. Si ves un endpoint que recibe `empresaId` “suelto”, está mal diseñado.

¿Por qué el operador se crea solo por consola? (lo preguntaste — aquí está la respuesta completa)

Un operador puede **acreditar pagos**, es decir, crear saldo de la nada. Quien decide que exista otro operador es el **dueño del servidor**, y su credencial para hacerlo es tener acceso a la consola del equipo — exactamente quien debe tenerla. Si existiera una pantalla de “alta de operador”, el robo de una sesión de operador se convertiría en el robo permanente de la plataforma. El código está en `Facturacion.Server/Modules/Operador/Auth/OperadoresCli.cs`.

6. Autorización por permisos

No se autoriza por rol; se autoriza por **permiso**. Los permisos son exactamente estos seis:

`timbrar, cancelar, administrar_usuarios, comprar_timbres, ver_reportes, configurar_empresa`

- Cada permiso es una **política de ASP.NET Core** (`RequireClaim("perm", "...")`).

- Los endpoints se anotan con la política: ej. `POST /api/productos` exige `configurar_empresa`.
- En el panel del operador hay las mismas seis políticas para las operaciones sensibles (`PolíticasDeOperador`): crear paquetes exige `configurar_empresa`, acreditar compras exige `comprar_timbres`, gestionar operadores exige `administrar_usuarios`.
- **“Ocultar un botón no es proteger”**. El cliente replica las políticas solo para dibujar la UI; el **Server** rechaza igual la operación aunque alguien llegue a la ruta a mano.

7. Cómo se guardan los datos (base de datos y aislamiento)

7.1 SQL Server + EF Core 10

- La base se crea y actualiza con **migraciones EF Core** (`dotnet ef database update`).
- Las migraciones también crean **procedimientos almacenados** clave: `ReservarFolio`, `ReservarTimbre`, `ConfirmarTimbre`, `DevolverTimbre`, `AcreditarCompra`.

7.2 Modelo en tres niveles

Cuenta -- 1 a N --> Empresa emisora --> Clientes, Productos, Series de folios, Comprobantes, BolsaTimbres, UsuariosEmpresas

Más los catálogos del SAT (`c_ClaveProdServ`, `c_CodigoPostal`, ...). Las tablas de Identity (`AspNetUsers`) y los catálogos del SAT **no llevan EmpresaId**.

7.3 Aislamiento por empresa — la joya de la casa

Se implementa en **tres capas** que trabajan juntas:

1. **El claim del token es la única fuente** (`ContextoEmpresaHttp`): cada petición lee `EmpresaId` del JWT. No hay otro camino.
2. **Filtro global de lectura** (`FiltroDeEmpresa`): con EF Core, toda entidad que implementa `IEntidadDeEmpresa` recibe automáticamente un `HasQueryFilter(...)` que dice `EmpresaId == empresa-actual-del-request`. Nadie tiene que acordarse de filtrar; el acceso a datos de otra empresa **ni siquiera aparece en la consulta SQL**. Si no hay empresa activa, el filtro “falla cerrado”: devuelve **cero renglones, nunca de más**.
3. **Interceptor de escritura** (`SelladoDeEmpresaInterceptor`): el filtro solo protege la lectura. Para escritura, antes de cada `SaveChanges` se sella `EmpresaId` en todo renglón nuevo (o lanza excepción si intenta meterse en otra empresa) y **bloquea** que un renglón existente cambie de empresa.

El panel del **operador** necesita ver datos de todas las empresas (acreditar una compra de cualquier cuenta). Por eso sus consultas usan `IgnoreQueryFilters()` **siempre acotadas con un filtro explícito** por cuenta/empresa y con justificación escrita en el código. No es un agujero: es el otro lado del mismo modelo.

7.4 Reglas de negocio sobre los datos

- **Dinero:** `decimal(18,6)` en la base. Cálculo con 6 decimales, presentación con 2. Nunca `double`, nunca `float`.
- **Fechas:** siempre UTC en la base; se convierten al mostrar según el huso de la empresa.
- **Nada se borra:** solo `Activo = false`. Única excepción: un borrador nunca timbrado.
- **Folios sin huecos ni concurrencia:** se apartan con `UPDATE ... WITH (UPDLOCK)` dentro de una transacción corta en el procedimiento `ReservarFolio`. Nunca `SELECT MAX(Folio)+1`. Si el timbrado falla después de tomar folio, el comprobante queda en **error** con ese folio apartado; **no se recicla** (una factura anulada con folio perdido es correcto fiscalmente).
- **La bolsa de timbres** también usa `UPDLOCK`: dos hilos no pueden dejar el saldo en negativo (procedimientos `ReservarTimbre`, `ConfirmarTimbre`, `DevolverTimbre`).
- **Inmutabilidad del comprobante:** al timbrar se copian **dentro del comprobante** los datos fiscales de emisor, receptor y conceptos. Que un cliente cambie su domicilio hoy no puede alterar una factura de hace dos años: los reportes leen las copias, no los catálogos.
- **Índices únicos con filtro:** el RFC de cliente es único *por empresa* y excluye los genéricos del SAT (`XAXX010101000`, `XEXX010101000`).
- **Bitácora:** toda operación que cambie datos fiscales deja registro con usuario, empresa, momento y valores anterior y nuevo.

7.5 Catálogos del SAT

`c_ClaveProdServ` (decenas de miles de filas) y `c_CodigoPostal` (cientos de miles) **no viajan al cliente** (serían MB en un navegador). Se buscan con un autocompletado contra el servidor (mínimo 3 caracteres, 300 ms de debounce, máx. 50 resultados, con índice de texto completo en SQL Server). Los catálogos chicos y estables se precargan y se cachean en el cliente. Una clave de catálogo **nunca se teclea a mano libre**.

8. Criptografía y secretos

Aquí está lo que más se pregunta en una revisión de seguridad:

Contraseñas

- Se guardan con **ASP.NET Core Identity** (`IPasswordHasher`): PBKDF2 con salt aleatorio. Nunca en claro, nunca comparables por consulta.

Certificados de sello digital (CSD) — .cer y .key

- Se guardan **cifrados** con **Data Protection** de ASP.NET Core y una **clave maestra** que vive **fuera del repositorio** (env var `Almacen_LlaveMaestraPfx`, un certificado base64).
- La cadena es: Protector de secretos → Data Protection → clave maestra (PFX). Los archivos quedan fuera de `wwwroot` en una carpeta (`Almacen_Raiz`) y se sirven por endpoint autorizado.
- Si se pierde la clave maestra → los CSD cargados quedan **ilegibles para siempre** y cada empresa debe volver a cargar su certificado. Por eso se respalda fuera del servidor.
- `appsettings.Development.json` **no se publica** (`CopyToPublishDirectory="Never"`): contenía la llave maestra en claro y viajaba dentro del artefacto. Fue el hallazgo del repaso de seguridad de la fase 9.

Token JWT

- La clave de firma (`Jwt_ClaveDeFirma`) vive en variable de entorno, mínimo 32 caracteres. El arranque **falla a propósito** si falta o es corta.

Base de datos: ¿encriptada?

Todo lo sensible se cifra **antes** de llegar a la base (contraseñas, CSD). La conexión a SQL Server usa la cadena de conexión (`ConnectionStrings__BaseDeDatos`); en producción se recomienda:

- `Encrypt=True` en la cadena para encriptar el tráfico de red hacia la base,
- y **TDE (Transparent Data Encryption)** a nivel de SQL Server si se quiere el disco del servidor de base de datos cifrado — es una decisión de infraestructura, no del código.

Correo

- El SMTP es **del SaaS**, no de los clientes. No se custodien contraseñas de correo de terceros — eso era del sistema viejo y en un multitenant significa guardar contaminados decenas de claves.

Cabeceras de seguridad

Todas las respuestas llevan `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`, HSTS. La CSP de una PWA con WebAssembly necesita `wasmedge-unsafe-eval` y no se afloja más que eso.

Idempotencia (billetera)

Todo POST que cobre o timbre exige el encabezado `Idempotency-Key`. El servidor guarda la clave con su respuesta 24 h; si llega la misma clave, devuelve **la misma respuesta** y no cobra dos veces. Protege contra dobles clics y reintentos de red.

Errores

Respuestas en **Problem Details (RFC 7807)** con `traceId`. El detalle de una excepción nunca se filtra al cliente; el `traceId` sí, para darle soporte al contador.

9. Cómo se monta el servicio (despliegue)

Resumen operativo de `docs/DESPLIEGUE.md` (léelo completo antes de desplegar):

9.1 Publicar

```
dotnet publish Facturacion.Server -c Release -o ./publicado
```

El publicado del Server ya incluye la app Blazor compilada con Brotli.

9.2 Variables de entorno obligatorias

(Sin ellas **no arranca**; se nombra el valor pero no se escribe aquí)

Variable	Qué es
ASPNETCORE_ENVIRONMENT	Production (apaga el sembrado, enciende comprobaciones estrictas)
ConnectionStrings__BaseDeDatos	Cadena de conexión a SQL Server
Jwt__Emisor, Jwt__Audiencia, Jwt__ClaveDeFirma	Token JWT (la clave >= 32 caracteres)
Almacen__LlaveMaestraPfx	Clave maestra (se genera con <code>--generar-llave-maestra</code>)
Correo__Servidor	SMTP del SaaS (es obligatorio fuera de Development)

Recomendadas: `Almacen__Raiz` y `Almacen__RutaLlaverero` (carpetas persistentes **fuera de `wwwroot`**), credenciales SMTP, correo/telefono de soporte.

9.3 Base de datos

```
dotnet ef database update --project Facturacion.Server
```

Crea las tablas, los procedimientos almacenados y los índices de texto completo. **No** hay migración automática al arrancar (dos instancias migrando a la vez romperían la base). Es un paso del despliegue, antes de arrancar.

9.4 Catálogos del SAT (imprescindible)

El archivo pesa ~48 MB y **no está en el repo** (es del SAT). Descárgalo y cárgalo:

```
dotnet run --project Facturacion.Server -- --cargar-catalogos ruta/al/catCFDI_V_4_AAAAMDD.x
```

Tarda varios minutos (~52 000 claves de producto, 95 000 CP, 145 000 colonias). Sin esto el sistema arranca pero **rechaza toda alta de cliente y producto**.

9.5 Esquemas y XSLT del SAT (obligatorio para timbrar)

En CatalogosSAT/: cfdv40.xsd, tdCFDI.xsd, catCFDI.xsd, cadenaoriginal_4_0.xslt y los **33 XSLT de complemento** (todos, aunque el MVP no los use: la cadena los incluye por URL). El sistema **no sale a internet** a buscarlos (una llamada de red dentro del sellado pondría al SAT en el camino crítico de cada timbrado).

9.6 Primer usuario

No hay sembrado en producción ni pantalla de alta de cuentas todavía. La primera cuenta + su primera empresa + su primer usuario se crean directo en la base; de ahí en adelante el resto entra por invitación desde la pantalla de usuarios. Cuando exista el registro público, esto se reemplaza.

9.7 Verificación de Brotli

```
curl -s -o /dev/null -D - -H "Accept-Encoding: br" https://tu-dominio/_framework/blazor.web
```

Debe responder content-encoding: br. Si un proxy inverso está delante, verifica que no reescriba Accept-Encoding. Es el error de despliegue más común; si se sirve sin compresión, la descarga inicial se triplica.

10. Comandos útiles del servidor

Ver Facturacion.Server/Program.cs, que tiene una sección de comandos antes de arrancar:

```
# 1) Generar la clave maestra (imprime un certificado en base64; RESPÁLDALO)  
dotnet run --project Facturacion.Server -- --generar-llave-maestra
```

```
# 2) Cargar los catálogos del SAT (48 MB, minutos)  
dotnet run --project Facturacion.Server -- --cargar-catalogos ruta/al/catCFDI.xls
```

3) Crear un operador del SaaS (consola obligatoria)

```
dotnet run --project Facturacion.Server -- --crear-operador operador@midominio.mx "Nombre de
```

4) Compras pendientes y acreditación manual

```
dotnet run --project Facturacion.Server -- --compras-pendientes
```

```
dotnet run --project Facturacion.Server -- --acreditar-compra <id-de-la-compra>
```

5) Migraciones y desarrollo

```
dotnet ef database update --project Facturacion.Server
```

```
dotnet build Facturacion.sln
```

```
dotnet run --project Facturacion.Server
```

11. Recorridos de extremo a extremo

11.1 Alta de un operador (tus preguntas respondidas paso a paso)

1. El comando:

```
dotnet run --project Facturacion.Server -- --crear-operador operador@midominio.mx "Luis
```

2. Valida el correo y el nombre. Normaliza el correo a mayúsculas (CorreoNormalizado).

3. Revisa que no exista ya un operador con ese correo (acaba con error si sí).

4. **Genera una contraseña aleatoria** de 24 caracteres, de un alfabeto sin caracteres que se confundan visualmente (nada de “1 vs l”, “0 vs O”):

```
const string alfabeto = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ23456789#%+=-";
RandomNumberGenerator.GetString(alfabeto, 24);
```

5. Arma el OperadorPlataforma: Activo = true, y **nace con los seis permisos** (quien crea un operador por consola es el dueño del SaaS; los permisos se ajustan después desde el panel con administrar_usuarios).

6. **Hashea la contraseña** con IPasswordHasher<OperadorPlataforma> (PBKDF2) y guarda solo el hash. La contraseña en claro **no se guarda ni se envía por correo**.

7. SaveChanges() lo inserta en OperadoresPlataforma.

8. Imprime la contraseña **una sola vez** en la consola: “Entrégala por un medio seguro”.

11.2 El operador inicia sesión y acredita una compra

```
POST /api/operador/auth/iniciar-sesion → valida credenciales → JWT (opr, perm:*)
                                         + cookie de refresh_op (rotativa)
```

GET /api/operador/compras	→ lista compras de todas las cuentas pendientes
POST /api/operador/compras/{id}/acreditar	→ exige permiso comprar_timbres → invoca dbo.AcreditarCompra (transaccional, idempotente) → la bolsa de la empresa recibe los timbres

11.3 El inquilino emite y timbra una factura

1. Formulario de emisión → borrador (POST /api/documentos/borradores)
2. El motor de impuestos calcula IVA/IEPS/ISH con decimal(18,6).
3. Guardar → borrador.
4. Timbrar (POST /api/documentos/{id}/timbrar) → Idempotency-Key obligatoria.
Se reserva folio (ReservarFolio, UPDLOCK) y timbre (ReservarTimbre).
5. Se sellan los datos fiscales copiados dentro del comprobante.
6. El XML se valida contra los XSD del SAT y se firma con el CSD de la empresa.
7. Se envía al PAC (SW Sapient) → PAC devuelve el timbre fiscal.
8. Se confirma el timbre (ConfirmarTimbre) → estatus "timbrado".
9. Se genera el PDF con QR y cadena original. Listo para descargar o enviar por correo.
Si algo falla → "error" con el folio apartado (no se recicla).

12. Lo que quedó fuera del MVP (fase 2)

Carta Porte, Comercio Exterior, Notaría, Constructoras, Cotizaciones, Addendas, complemento INE, inventario. Si están modelados los campos de licencia por empresa (LicNotarios, LicObras, LicComercio, LicINE), para que la fase 2 sea agregar módulos y no rehacer el esquema.

13. Glosario

Término	Significado
CFDI 4.0	Comprobante Fiscal Digital por Internet, versión vigente del SAT
PAC	Proveedor Autorizado de Certificación; timbra (da validez fiscal) el XML
CSD	Certificado de Sello Digital: .cer + .key para firmar el XML
Cadena original	Texto transformado del XML por XSLT; se firma con el CSD
Carta Porte / IEPS	Complementos y impuestos especiales (fase 2 / motor de impuestos)
Tenencia	A qué empresa pertenece cada dato

Término	Significado
Query filter	Filtro que EF Core inyecta en cada SELECT por EmpresaId
Interceptor	Hooks de EF Core antes de guardar (aquí: sellar EmpresaId)
UPDLOCK	Bloqueo de SQL Server para actualizar un renglón sin carreras
Idempotencia	Repetir la misma operación con la misma clave produce el mismo resultado
Sembrado	Datos de prueba iniciales; solo corre en Development